HUNDRED DAYS OF CODE
DAY 34

QUESTION: Write a C program to perform the following menu-driven operations using separate functions:

1. Read three variable-length 2D arrays and verify the left distributive property: A*(B+C) = (A*B) + (A*C). The dimensions of input matrices are given as follows: A (m x n), B (n x p), C (n x p).
2. Read three variable-length 2D arrays and verify the right distributive property: (A+B)*C = (A*C) + (B*C). The dimensions of input matrices are given as follows: A (m x n), B (m x n), C (n x p)
3. Exit

SOLUTION:

1. Aim

To write a menu-driven C program using **separate functions and variable-length 2D arrays** to verify:

1. **Left distributive property:** A × (B + C) = (A × B) + (A × C)
2. **Right distributive property:** (A + B) × C = (A × C) + (B × C)
3. Exit the program.

---

2. Algorithm

Main Algorithm

1. Start the program.
2. Display the menu:
    - Verify left distributive property.
    - Verify right distributive property.
    - Exit.
3. Accept the user's choice.
4. Use a switch statement to call the appropriate function.
5. Repeat the menu until the user selects **Exit**.
6. Stop.

HUNDRED DAYS OF CODE
DAY 34

Operation 1: Left Distributive Property

Given:

- A has dimensions m × n
- B has dimensions n × p
- C has dimensions n × p

Steps:

1. Input m, n, and p.
2. Declare variable-length matrices A[m][n], B[n][p], and C[n][p].
3. Input the elements of all three matrices.
4. Calculate B + C.
5. Multiply A with (B + C) to obtain the left-hand side.
6. Calculate A × B.
7. Calculate A × C.
8. Add (A × B) and (A × C) to obtain the right-hand side.
9. Compare the corresponding elements of both results.
10. If they are equal, the left distributive property is verified.

Operation 2: Right Distributive Property

Given:

- A has dimensions m × n
- B has dimensions m × n
- C has dimensions n × p

Steps:

1. Input m, n, and p.
2. Declare variable-length matrices A[m][n], B[m][n], and C[n][p].
3. Input the elements of all three matrices.

HUNDRED DAYS OF CODE
DAY 34

4. Calculate A + B.
5. Multiply (A + B) with C to obtain the left-hand side.
6. Calculate A × C.
7. Calculate B × C.
8. Add (A × C) and (B × C) to obtain the right-hand side.
9. Compare the two resulting matrices.
10. If all corresponding elements are equal, the right distributive property is verified.

C PROGRAM:

```
#include <stdio.h>

// Function declarations
void leftDistributive();
void rightDistributive();

int main()
{
  int choice;

  do
  {
    printf("\n----- MATRIX DISTRIBUTIVE PROPERTY MENU -----\n");
    printf("1. Verify Left Distributive Property\n");
    printf("2. Verify Right Distributive Property\n");
    printf("3. Exit\n");

    printf("Enter your choice: ");
```

HUNDRED DAYS OF CODE
DAY 34

```
    scanf("%d", &choice);

    switch(choice)
    {
        case 1:
            leftDistributive();
            break;

        case 2:
            rightDistributive();
            break;

        case 3:
            printf("Program exited successfully.\n");
            break;

        default:
            printf("Invalid choice! Please try again.\n");
    }

  } while(choice != 3);

  return 0;
}
```

HUNDRED DAYS OF CODE
DAY 34

```
// Function to verify Left Distributive Property
void leftDistributive()
{
   int m, n, p;
   int i, j, k;
   int equal = 1;

   printf("\nEnter m, n and p: ");
   scanf("%d %d %d", &m, &n, &p);

   // Variable Length Arrays
   int A[m][n];
   int B[n][p];
   int C[n][p];

   int BC[n][p];
   int AB[m][p];
   int AC[m][p];

   int left[m][p];
   int right[m][p];

   // Input Matrix A
   printf("\nEnter elements of Matrix A (%d x %d):\n", m, n);

   for(i = 0; i < m; i++)
```

HUNDRED DAYS OF CODE
DAY 34

```
{
   for(j = 0; j < n; j++)
      scanf("%d", &A[i][j]);
}

// Input Matrix B
printf("\nEnter elements of Matrix B (%d x %d):\n", n, p);

for(i = 0; i < n; i++)
{
   for(j = 0; j < p; j++)
      scanf("%d", &B[i][j]);
}

// Input Matrix C
printf("\nEnter elements of Matrix C (%d x %d):\n", n, p);

for(i = 0; i < n; i++)
{
   for(j = 0; j < p; j++)
      scanf("%d", &C[i][j]);
}

// Calculate B + C
for(i = 0; i < n; i++)
{
```

HUNDRED DAYS OF CODE
DAY 34

```
   for(j = 0; j < p; j++)
      BC[i][j] = B[i][j] + C[i][j];
}

// Calculate A x (B + C)
for(i = 0; i < m; i++)
{
   for(j = 0; j < p; j++)
   {
      left[i][j] = 0;

      for(k = 0; k < n; k++)
         left[i][j] += A[i][k] * BC[k][j];
   }
}

// Calculate A x B
for(i = 0; i < m; i++)
{
   for(j = 0; j < p; j++)
   {
      AB[i][j] = 0;

      for(k = 0; k < n; k++)
         AB[i][j] += A[i][k] * B[k][j];
   }
```

HUNDRED DAYS OF CODE
DAY 34

```
}

// Calculate A x C
for(i = 0; i < m; i++)
{
   for(j = 0; j < p; j++)
   {
      AC[i][j] = 0;

      for(k = 0; k < n; k++)
         AC[i][j] += A[i][k] * C[k][j];
   }
}

// Calculate (A x B) + (A x C)
for(i = 0; i < m; i++)
{
   for(j = 0; j < p; j++)
      right[i][j] = AB[i][j] + AC[i][j];
}

// Display Left Side
printf("\nA x (B + C):\n");

for(i = 0; i < m; i++)
{
```

HUNDRED DAYS OF CODE
DAY 34

```
    for(j = 0; j < p; j++)
        printf("%d\t", left[i][j]);

    printf("\n");
}

// Display Right Side
printf("\n(A x B) + (A x C):\n");

for(i = 0; i < m; i++)
{
    for(j = 0; j < p; j++)
        printf("%d\t", right[i][j]);

    printf("\n");
}

// Compare both results
for(i = 0; i < m; i++)
{
    for(j = 0; j < p; j++)
    {
        if(left[i][j] != right[i][j])
            equal = 0;
    }
}
```

HUNDRED DAYS OF CODE
DAY 34

```
    if(equal == 1)
    {
        printf("\nA x (B + C) = (A x B) + (A x C)\n");
        printf("Hence, Left Distributive Property is verified.\n");
    }
    else
    {
        printf("\nLeft Distributive Property is not verified.\n");
    }
}


// Function to verify Right Distributive Property
void rightDistributive()
{
    int m, n, p;
    int i, j, k;
    int equal = 1;

    printf("\nEnter m, n and p: ");
    scanf("%d %d %d", &m, &n, &p);

    // Variable Length Arrays
    int A[m][n];
    int B[m][n];
```

HUNDRED DAYS OF CODE
DAY 34

```
int C[n][p];


int AB[m][n];
int AC[m][p];
int BC[m][p];


int left[m][p];
int right[m][p];


// Input Matrix A
printf("\nEnter elements of Matrix A (%d x %d):\n", m, n);


for(i = 0; i < m; i++)
{
   for(j = 0; j < n; j++)
      scanf("%d", &A[i][j]);
}


// Input Matrix B
printf("\nEnter elements of Matrix B (%d x %d):\n", m, n);


for(i = 0; i < m; i++)
{
   for(j = 0; j < n; j++)
      scanf("%d", &B[i][j]);
}
```

HUNDRED DAYS OF CODE
DAY 34

```
// Input Matrix C
printf("\nEnter elements of Matrix C (%d x %d):\n", n, p);

for(i = 0; i < n; i++)
{
   for(j = 0; j < p; j++)
      scanf("%d", &C[i][j]);
}

// Calculate A + B
for(i = 0; i < m; i++)
{
   for(j = 0; j < n; j++)
      AB[i][j] = A[i][j] + B[i][j];
}

// Calculate (A + B) x C
for(i = 0; i < m; i++)
{
   for(j = 0; j < p; j++)
   {
      left[i][j] = 0;

      for(k = 0; k < n; k++)
         left[i][j] += AB[i][k] * C[k][j];
```

HUNDRED DAYS OF CODE
DAY 34

```
    }
}

// Calculate A x C
for(i = 0; i < m; i++)
{
    for(j = 0; j < p; j++)
    {
        AC[i][j] = 0;

        for(k = 0; k < n; k++)
            AC[i][j] += A[i][k] * C[k][j];
    }
}

// Calculate B x C
for(i = 0; i < m; i++)
{
    for(j = 0; j < p; j++)
    {
        BC[i][j] = 0;

        for(k = 0; k < n; k++)
            BC[i][j] += B[i][k] * C[k][j];
    }
}
```

HUNDRED DAYS OF CODE
DAY 34

```
// Calculate (A x C) + (B x C)
for(i = 0; i < m; i++)
{
   for(j = 0; j < p; j++)
      right[i][j] = AC[i][j] + BC[i][j];
}

// Display Left Side
printf("\n(A + B) x C:\n");

for(i = 0; i < m; i++)
{
   for(j = 0; j < p; j++)
      printf("%d\t", left[i][j]);

   printf("\n");
}

// Display Right Side
printf("\n(A x C) + (B x C):\n");

for(i = 0; i < m; i++)
{
   for(j = 0; j < p; j++)
      printf("%d\t", right[i][j]);
```

HUNDRED DAYS OF CODE
DAY 34

```
    printf("\n");
  }

  // Compare both results
  for(i = 0; i < m; i++)
  {
    for(j = 0; j < p; j++)
    {
      if(left[i][j] != right[i][j])
        equal = 0;
    }
  }

  if(equal == 1)
  {
    printf("\n(A + B) x C = (A x C) + (B x C)\n");
    printf("Hence, Right Distributive Property is verified.\n");
  }
  else
  {
    printf("\nRight Distributive Property is not verified.\n");
  }
}
```

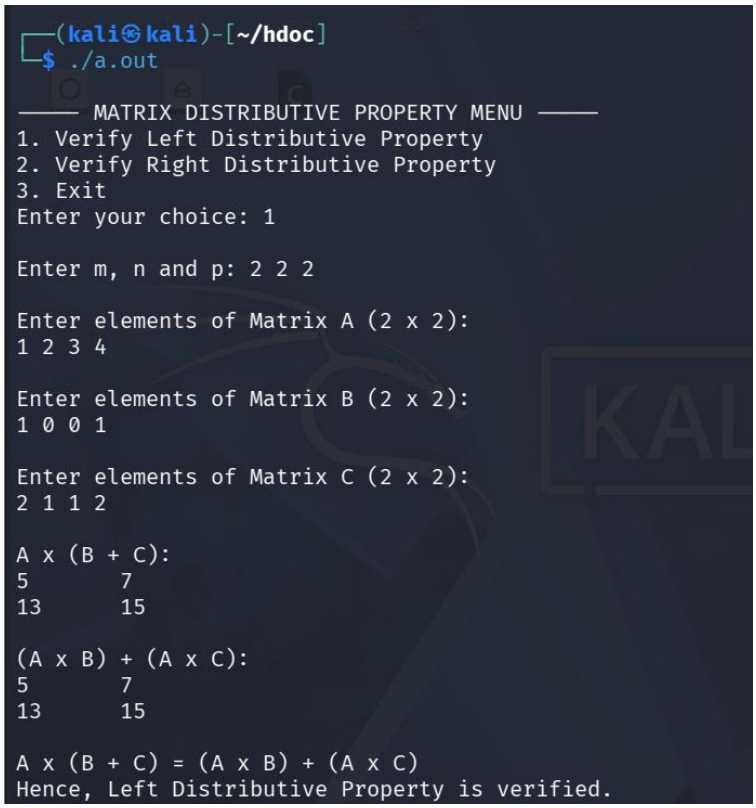
HUNDRED DAYS OF CODE
DAY 34

TEST CASE TABLE:

Test Case	Operation	Dimensions	Expected Result
1	Left distributive	A=`2×2`, B=`2×2`, C=`2×2`	Both sides are equal
2	Left distributive	A=`2×3`, B=`3×2`, C=`3×2`	Both results are 2×2 and equal
3	Left distributive	A=`1×2`, B=`2×1`, C=`2×1`	Both results are equal
4	Right distributive	A=`2×2`, B=`2×2`, C=`2×2`	Both sides are equal
5	Right distributive	A=`2×3`, B=`2×3`, C=`3×2`	Both results are 2×2 and equal
6	Right distributive	A=`1×2`, B=`1×2`, C=`2×1`	Both results are equal
7	Exit	Choice = 3	Program terminates
8	Invalid	Choice = 5	Displays invalid choice message

HUNDRED DAYS OF CODE
DAY 34

COMPILATION AND EXECUTION:



```
┌──(kali㉿kali)-[~/hdoc]
└─$ ./a.out

——— MATRIX DISTRIBUTIVE PROPERTY MENU ———
1. Verify Left Distributive Property
2. Verify Right Distributive Property
3. Exit
Enter your choice: 1

Enter m, n and p: 2 2 2

Enter elements of Matrix A (2 x 2):
1 2 3 4

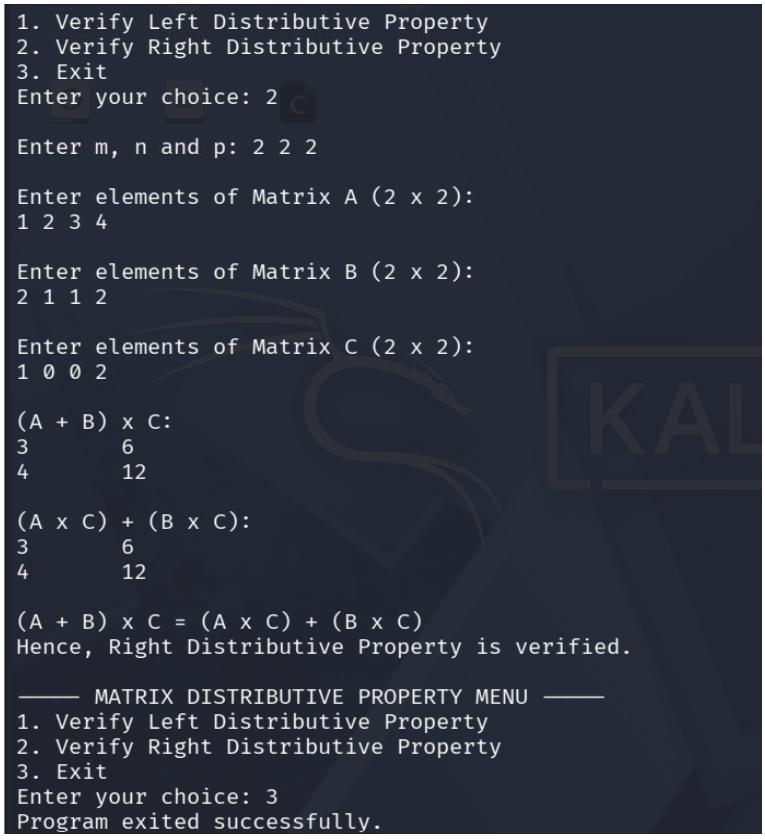
Enter elements of Matrix B (2 x 2):
1 0 0 1

Enter elements of Matrix C (2 x 2):
2 1 1 2

A x (B + C):
5       7
13      15

(A x B) + (A x C):
5       7
13      15

A x (B + C) = (A x B) + (A x C)
Hence, Left Distributive Property is verified.
```



```
1. Verify Left Distributive Property
2. Verify Right Distributive Property
3. Exit
Enter your choice: 2

Enter m, n and p: 2 2 2

Enter elements of Matrix A (2 x 2):
1 2 3 4

Enter elements of Matrix B (2 x 2):
2 1 1 2

Enter elements of Matrix C (2 x 2):
1 0 0 2

(A + B) x C:
3       6
4       12

(A x C) + (B x C):
3       6
4       12

(A + B) x C = (A x C) + (B x C)
Hence, Right Distributive Property is verified.

——— MATRIX DISTRIBUTIVE PROPERTY MENU ———
1. Verify Left Distributive Property
2. Verify Right Distributive Property
3. Exit
Enter your choice: 3
Program exited successfully.
```